# APPLICATION LOAD BALANCER

## Class no:21

## Application Load Balancer (ALB)

- **Definition**: An AWS service that distributes incoming application traffic across multiple targets (EC2, containers, IPs).
- **Protocol Used**: Works at **Layer 7 (Application Layer)** using **HTTP/HTTPS protocols**.
- **Intelligence**: It's "intelligent" because it can inspect the content of requests (headers, paths, hostnames) and make routing decisions based on rules.
- **Where Mostly Used**:
    - **Microservices architectures** → Different services (auth, catalog, payment) can be hosted separately and routed intelligently.
    - **Containerized applications** → Works seamlessly with ECS/EKS.

### Path-Based Routing Example

- ALB can route traffic based on **URL path**:
    - www.shop.com/auth/* → Authentication service
    - www.shop.com/catalog/* → Product catalog service

This allows each microservice to scale independently and be managed separately.

### Microservices vs Monolithic

| **Aspect** | **Monolithic Architecture** | **Microservices Architecture** |
|---|---|---|
| **Structure** | Single, large codebase | Multiple small, independent services |
| **Deployment** | Entire app deployed together | Each service deployed independently |
| **Scalability** | Scale whole app | Scale specific services |
| **Flexibility** | Harder to adopt new tech | Each service can use different tech |
| **Failure Impact** | One bug can crash whole app | Failure isolated to one service |

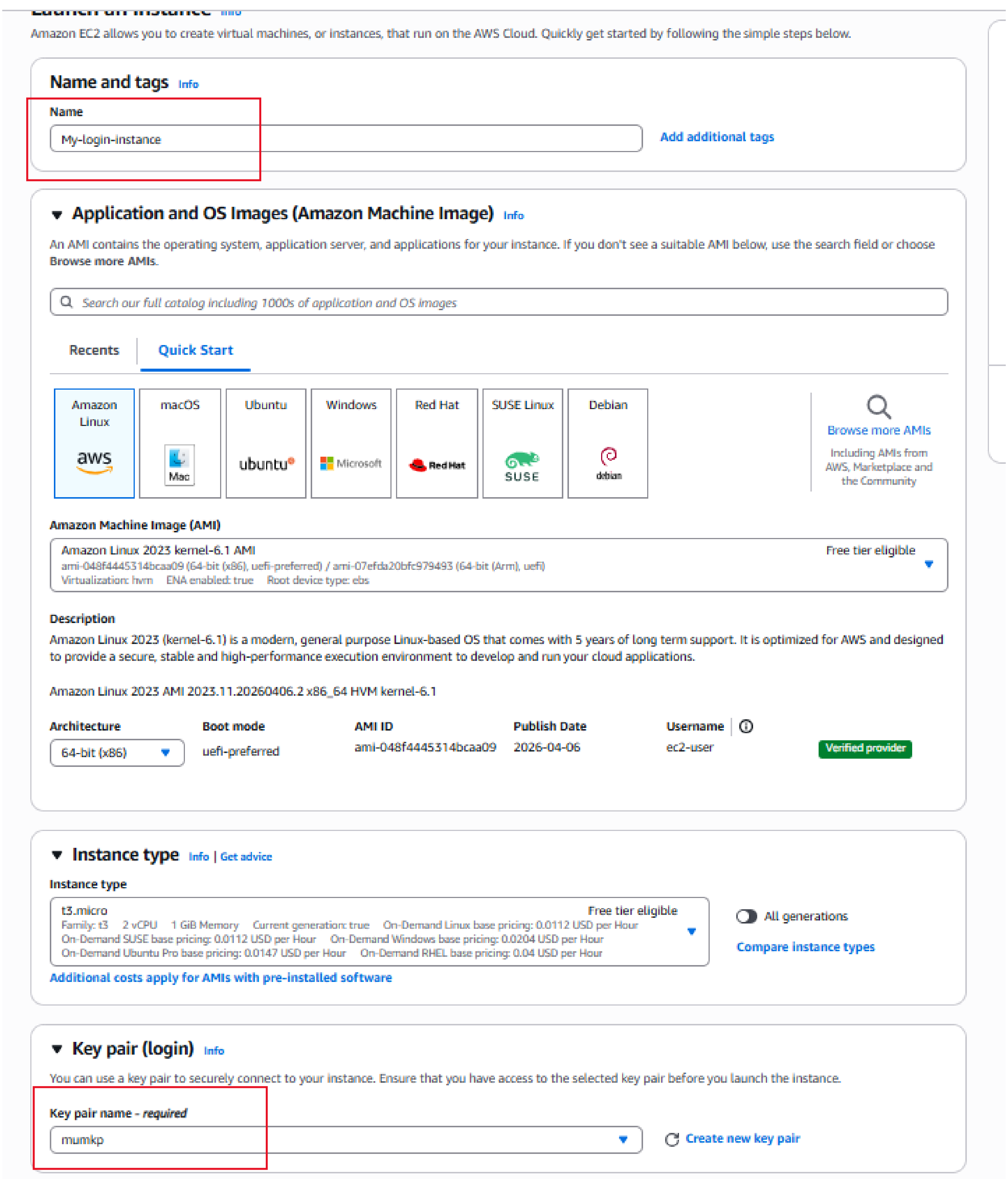
- **ALB** is crucial for modern **microservices** because it supports **path-based routing** and **content-based decisions** at the application layer.
- It uses **HTTP/HTTPS protocols** and provides intelligent traffic distribution.
- **Microservices** break applications into independent modules, unlike **monolithic** apps which are tightly coupled.

***<u>Hands on: Creating two instances for login and product, launching it using the ALB.</u>***

**<u>Step no: 1</u>** Create an EC2 instance for login,

Name: **My-login-instance**

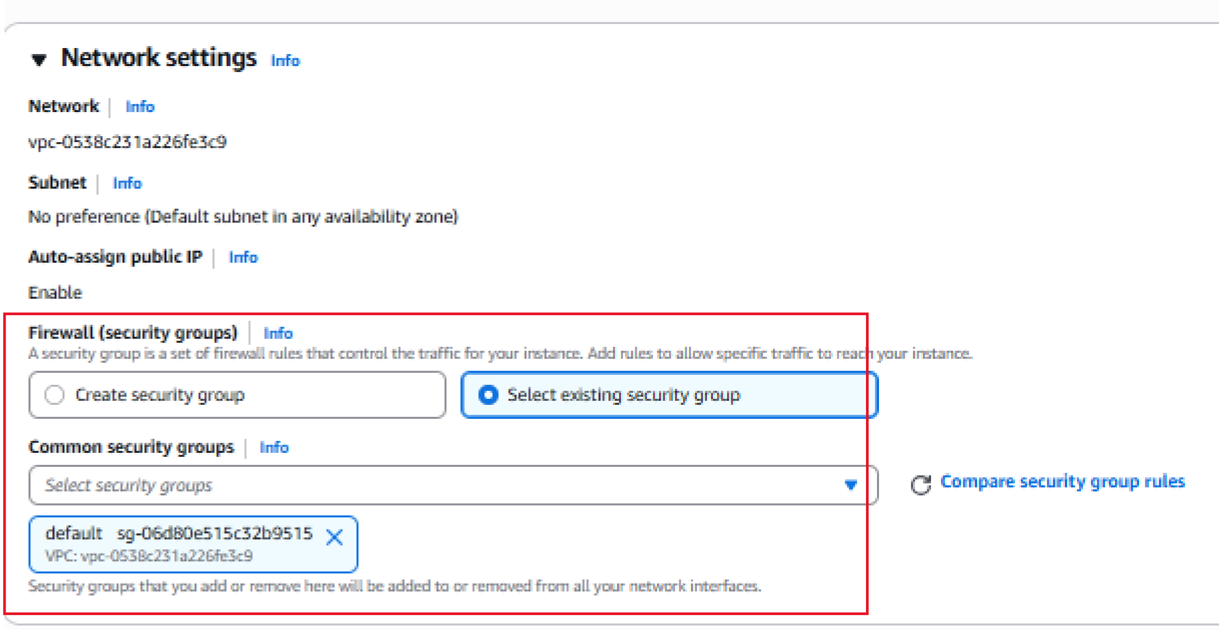
With OS type as Amazon Linux and with a key pair.



Network settings,

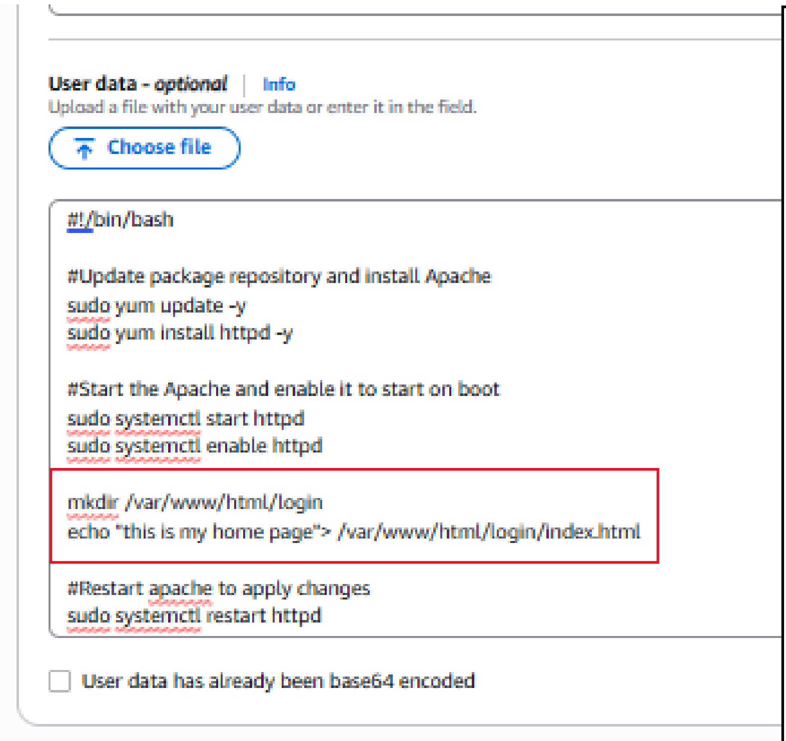
Choose: **Existing security group**

Choose: **default security group** (ensure http and https is enabled in the inbound rules)




Under **User data** → paste the comments given below

**Note:** for login mkdir /var/www/html/**login** and in echo "this is my home page"> /var/www/html/**login**/index.html

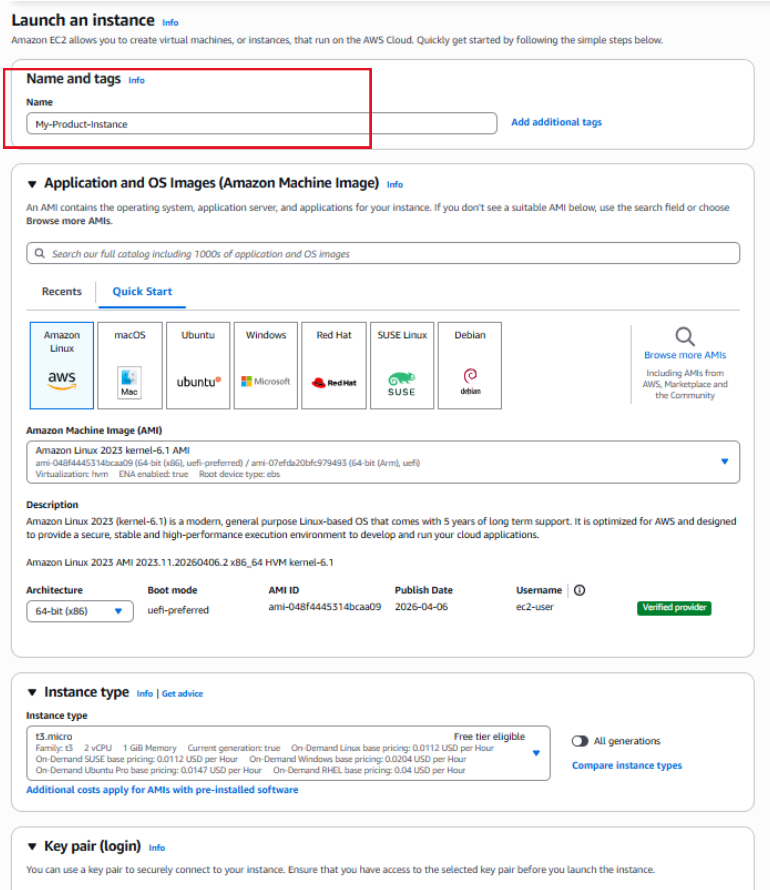



```
#!/bin/bash

#Update package repository and install Apache
sudo yum update -y
sudo yum install httpd -y

#Start the Apache and enable it to start on boot
sudo systemctl start httpd
sudo systemctl enable httpd

mkdir /var/www/html/login
echo "this is my home page">
/var/www/html/login/index.html

#Restart apache to apply changes
sudo systemctl restart httpd
```

**Step no: 2**Create an EC2 instance for product,

Name: **My-product-instance**

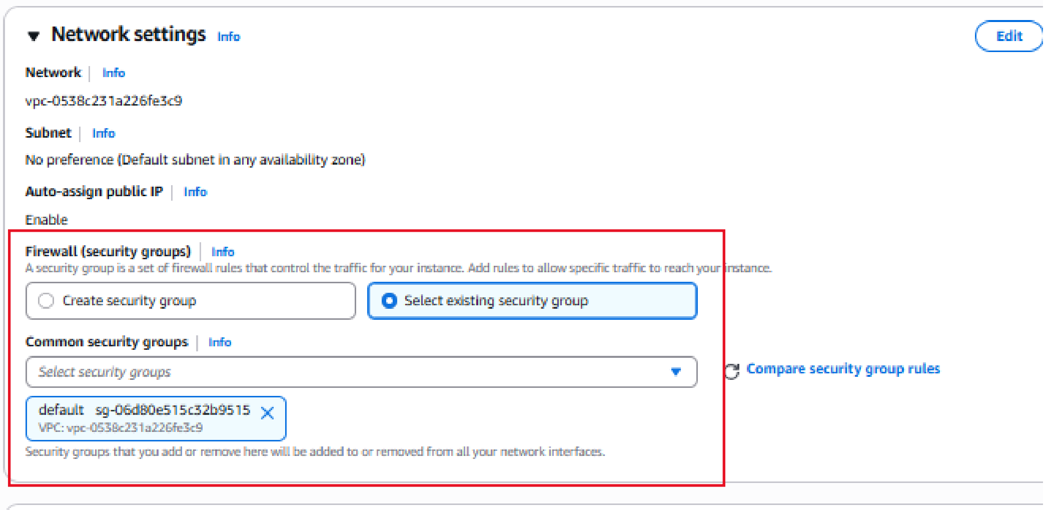
With OS type as Amazon Linux and with a key pair.




Network settings,

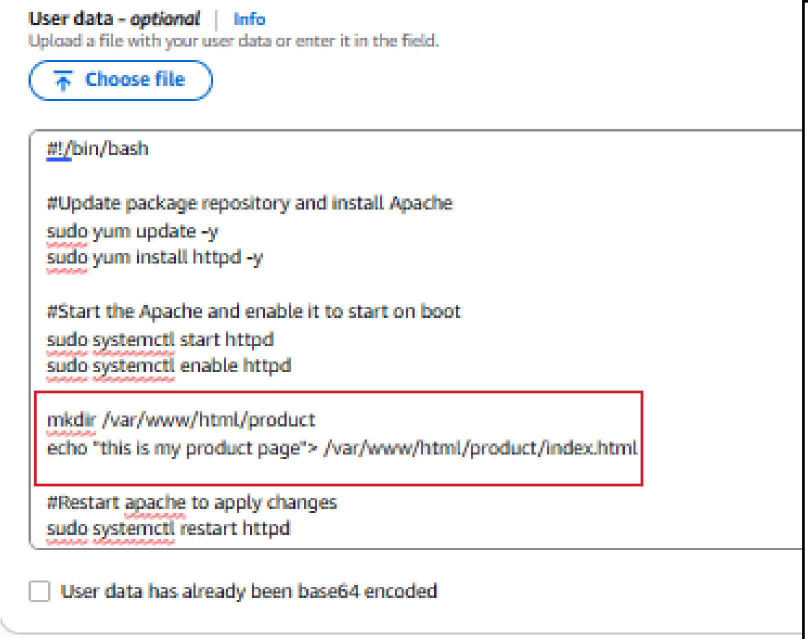
Choose: **Existing security group**

Choose: **default security group** (ensure http and https is enabled in the inbound rules)



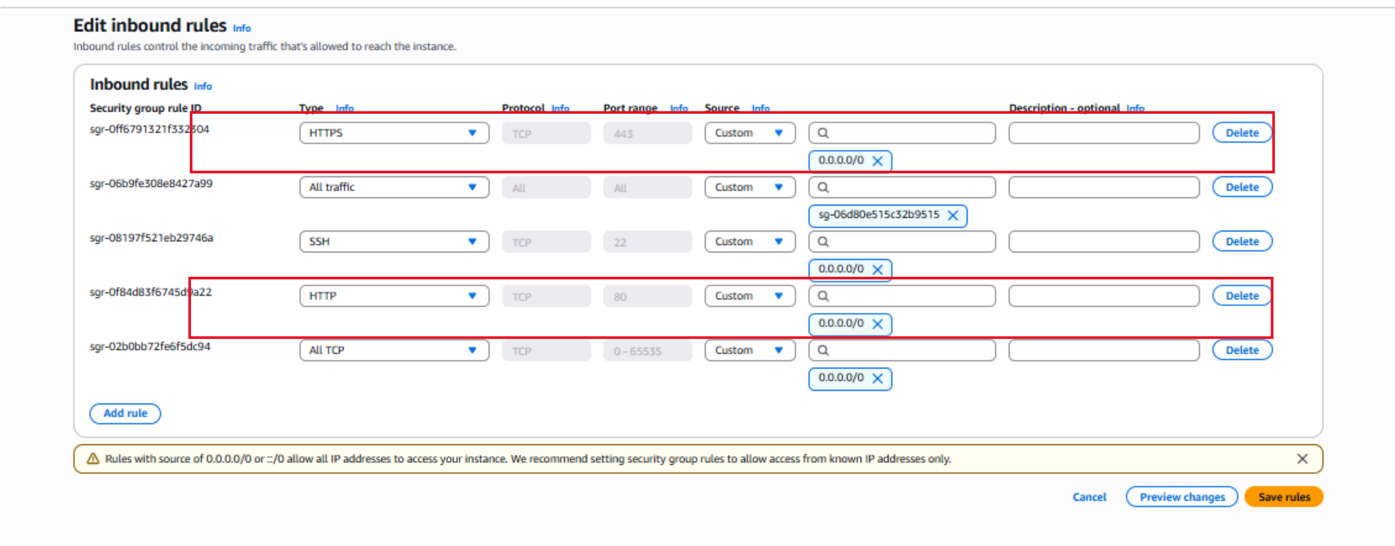
Under **User data** → paste the comments given below

**Note:** for login mkdir /var/www/html/**product** and in echo "this is my home page"> /var/www/html/**product**/index.html




```
#!/bin/bash

#Update package repository and install Apache
sudo yum update -y
sudo yum install httpd -y

#Start the Apache and enable it to start on boot
sudo systemctl start httpd
sudo systemctl enable httpd

mkdir /var/www/html/product
echo "this is my home page">
/var/www/html/product/index.html

#Restart apache to apply changes
sudo systemctl restart httpd
```

**Step no:3** Go to the default security group → edit inbound rules → add http and https → save rules.

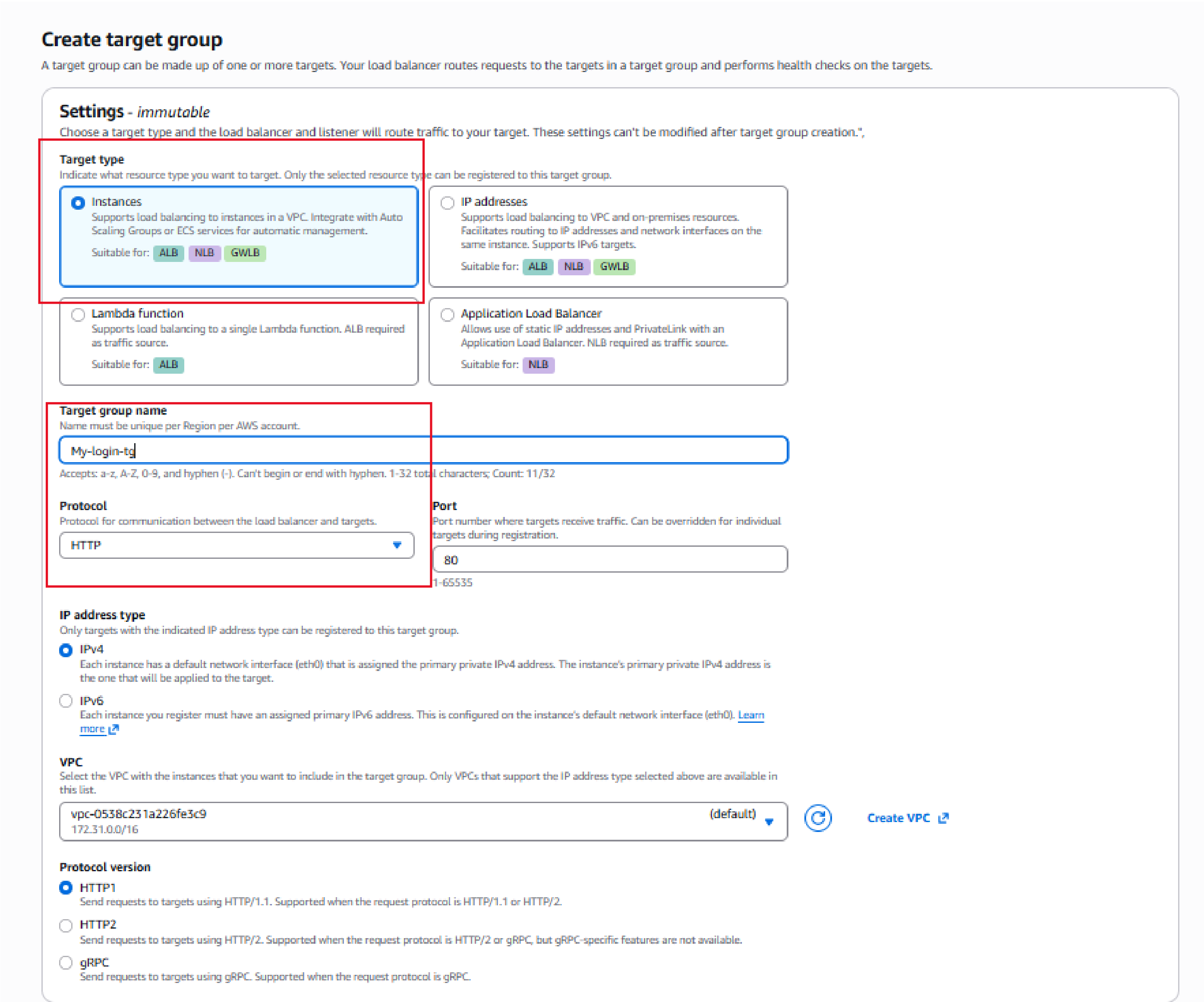


**Step no:4** Scroll down →go to target group → Create a target group for login instance.

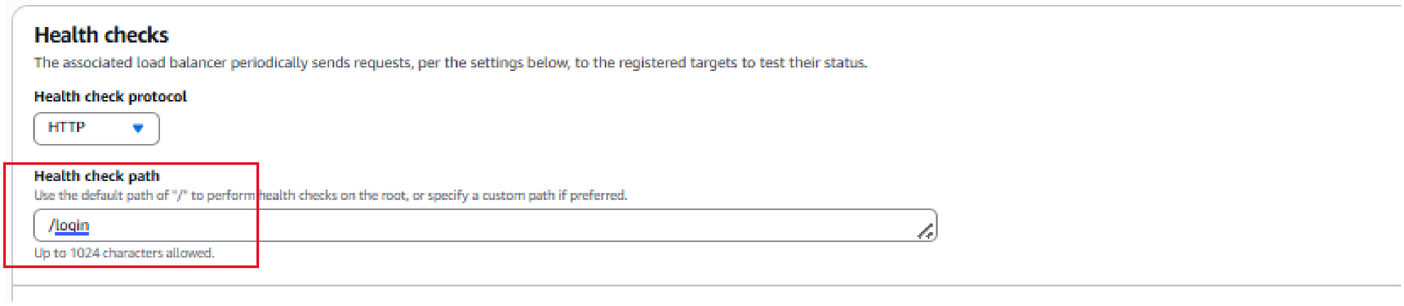
Choose: **Instances**

Name: **My-login-tg**

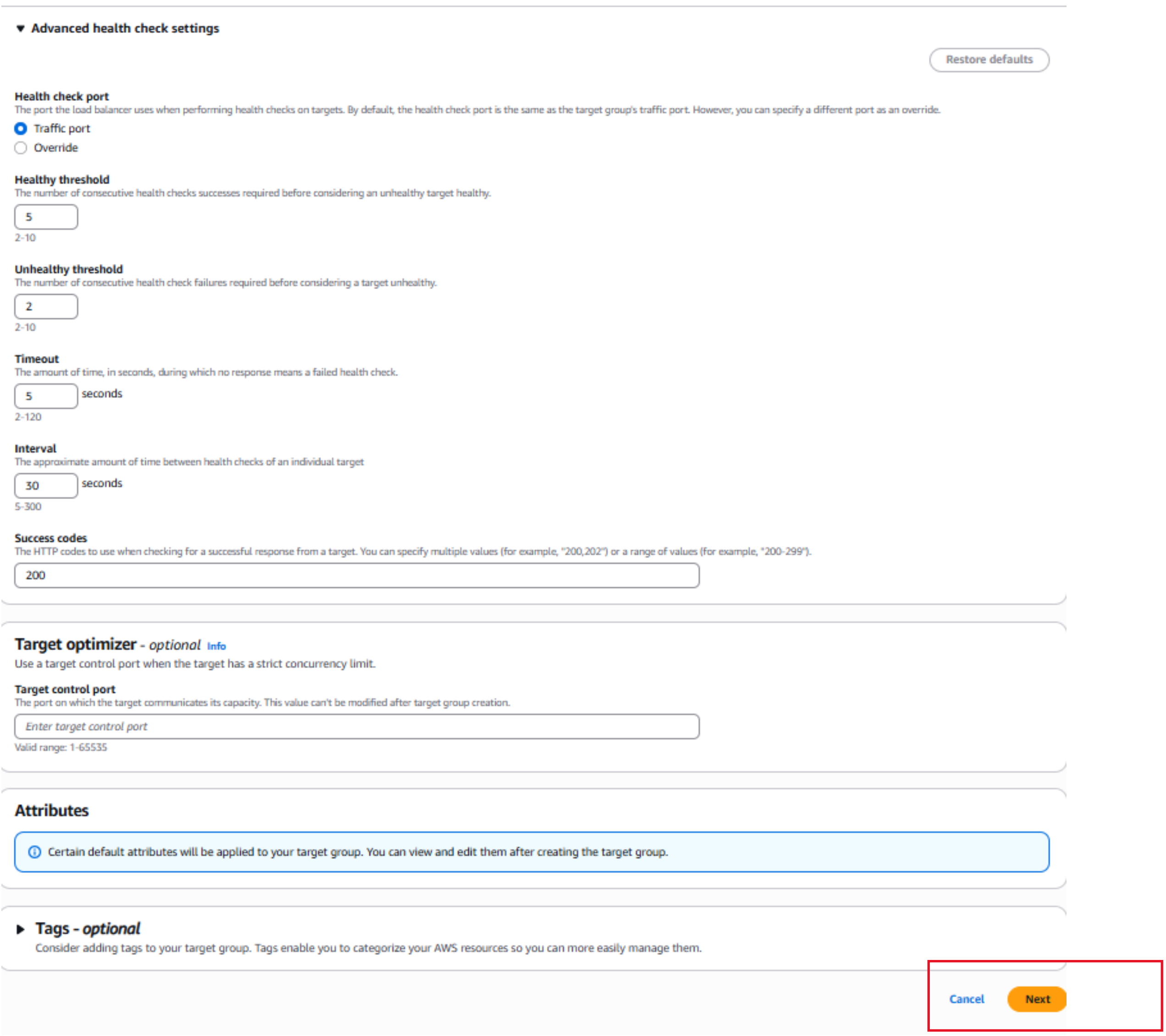
Protocol: **HTTP**



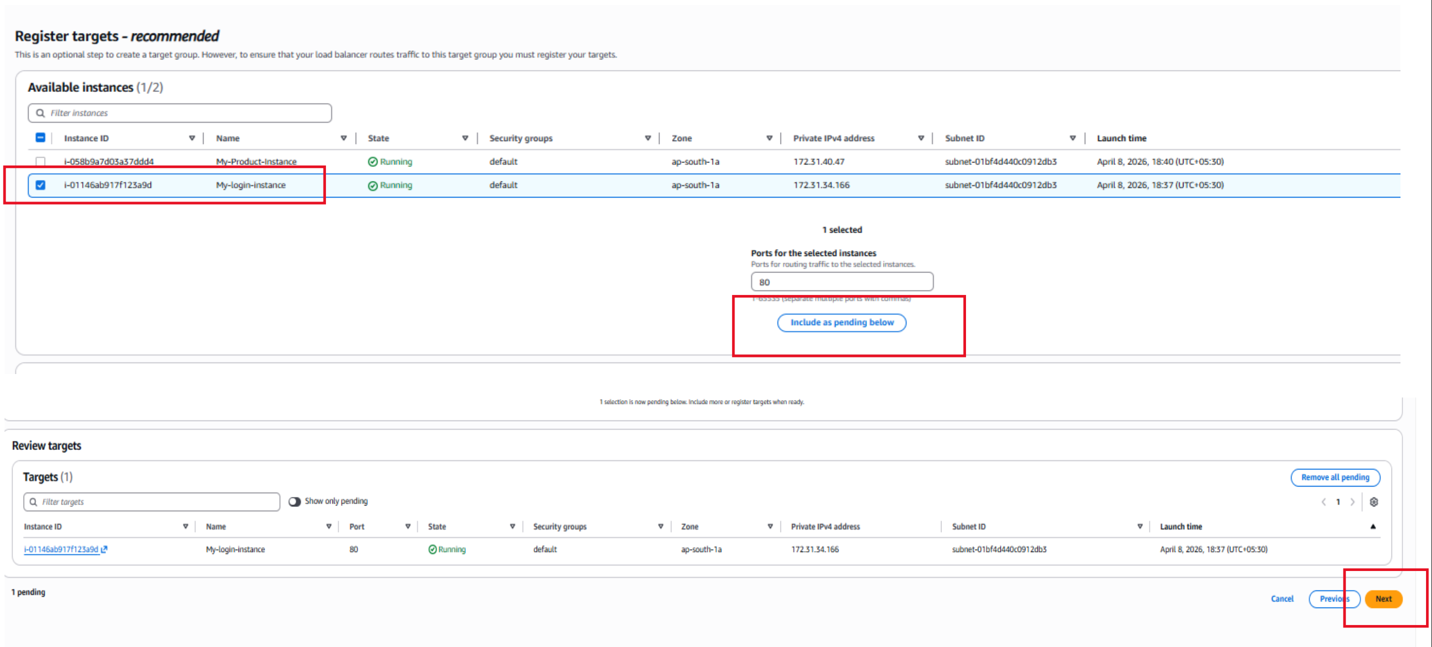

Health checks → http → Health check path as **/login** (because this target group is for login)



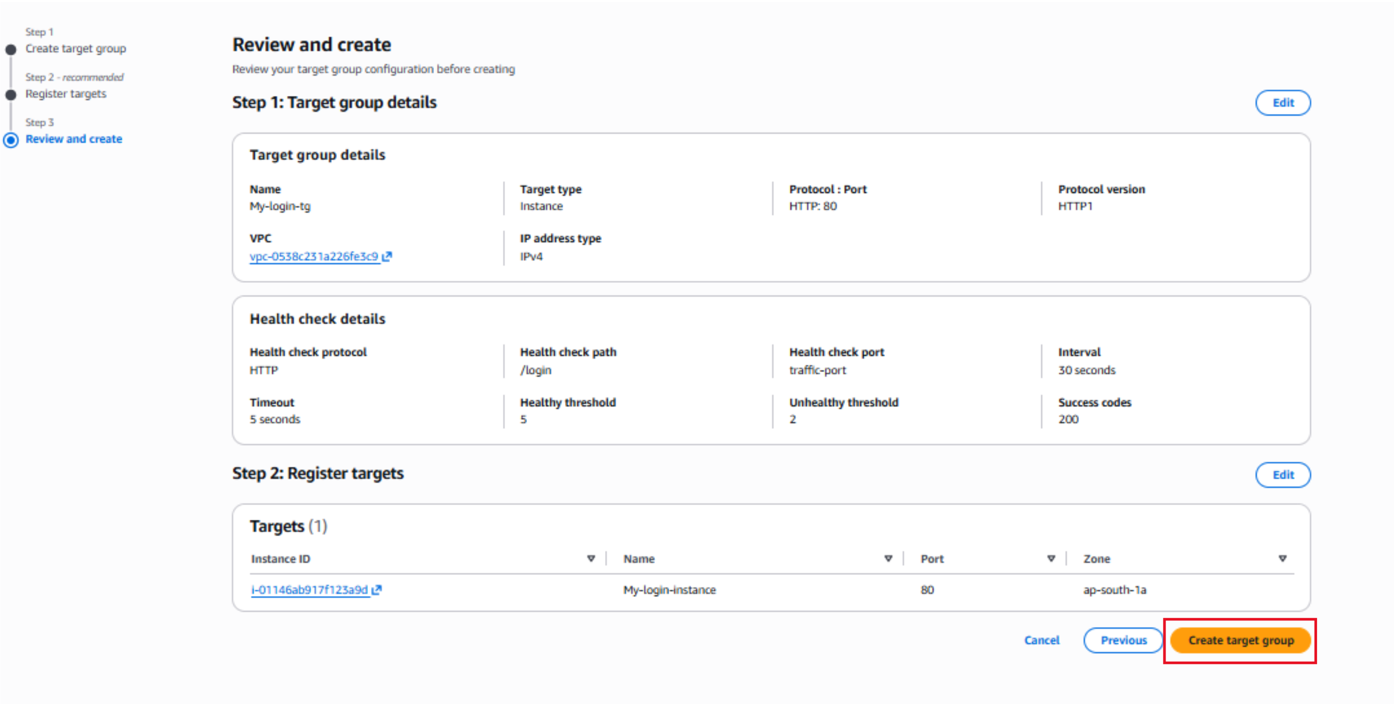
Keep the rest as default → click **Next.**




Choose **include as pending below → add the login instance → next.**



Click Create target group.



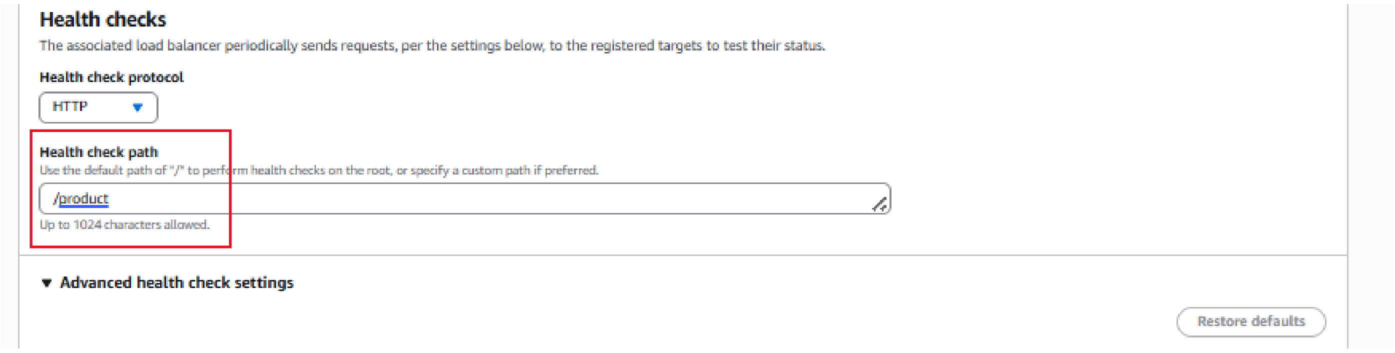

**Step no:5** Scroll down →go to target group → Create a target group for product instance.

Choose: **Instances**

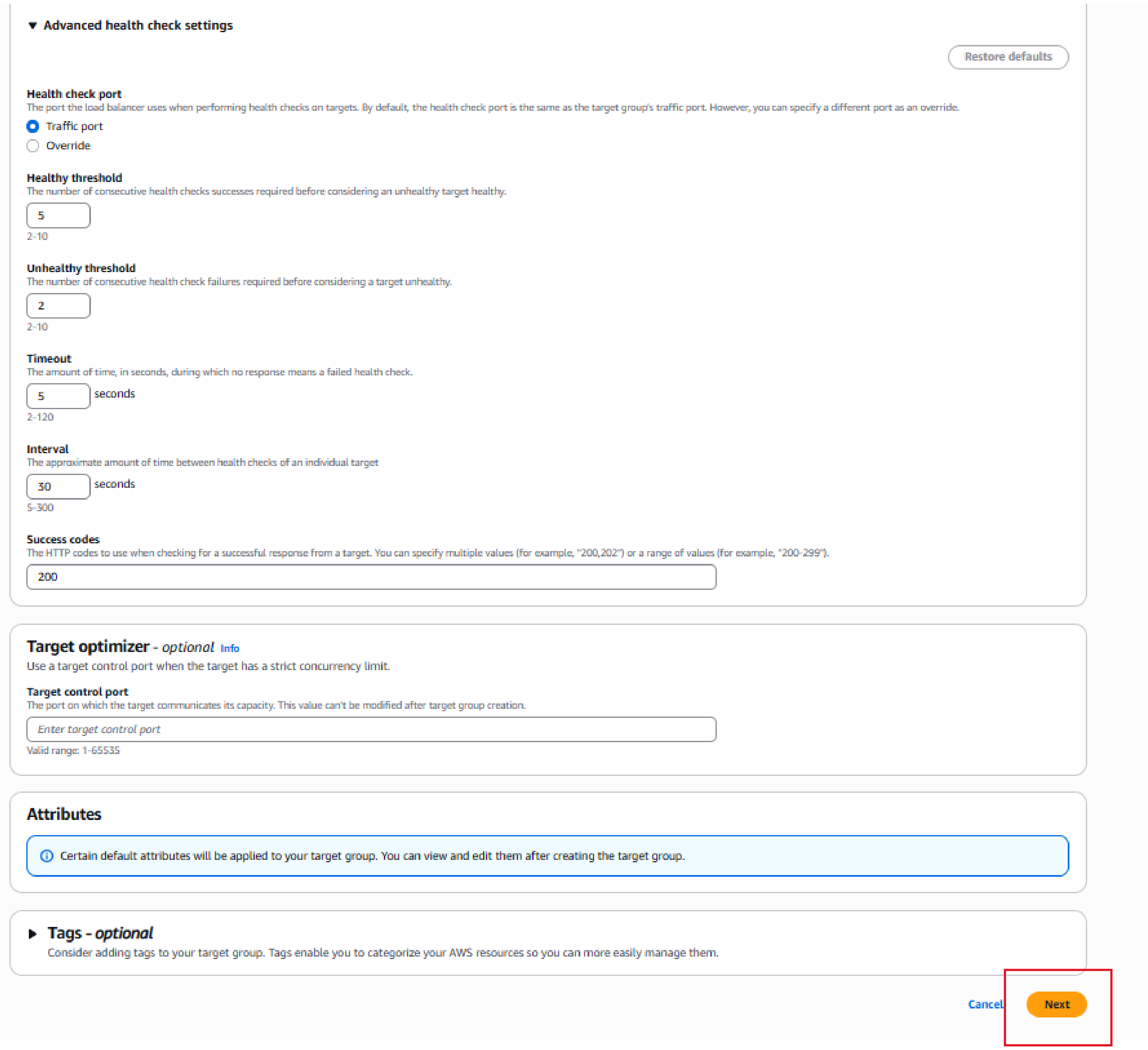
Name: **My-Product-tg**

Protocol: **HTTP**

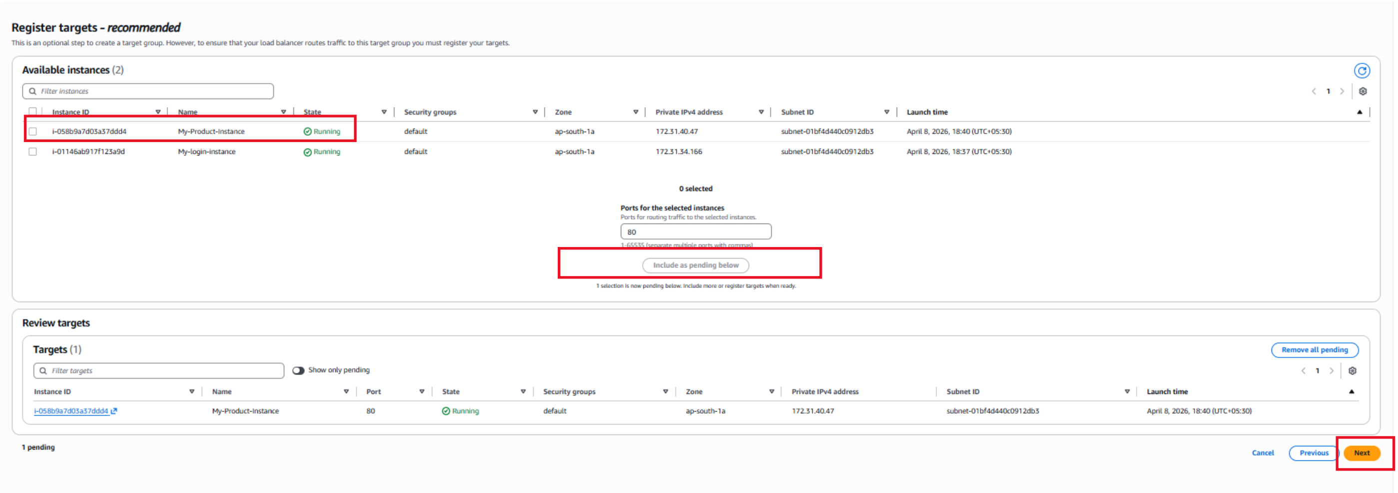
Health checks → http → Health check path as **/product** (because this target group is for product)



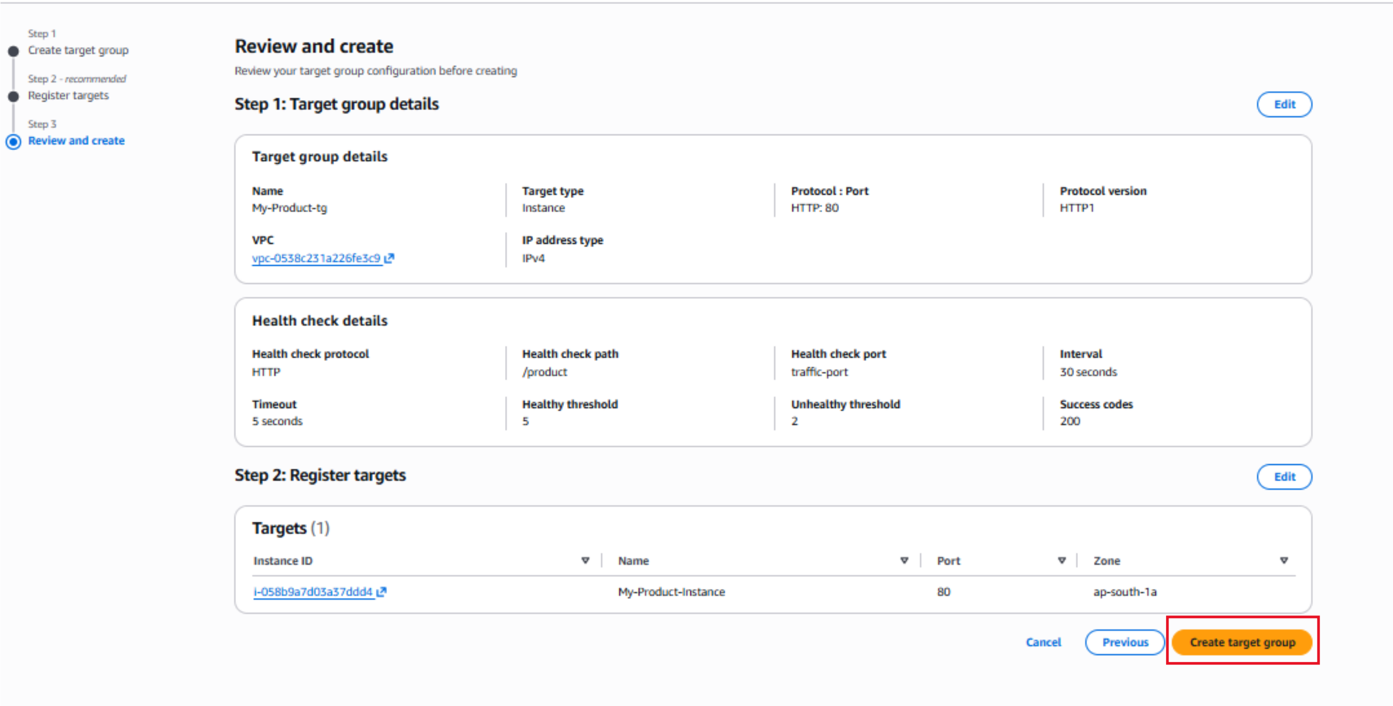

Keep the rest as default → click **Next.**



Choose **include as pending below → add the product instance → next.**




Click Create target group.




**<u>Step no:6</u>** Scroll down → Load balancer → Choose ALB.

Name: **My-ALB**

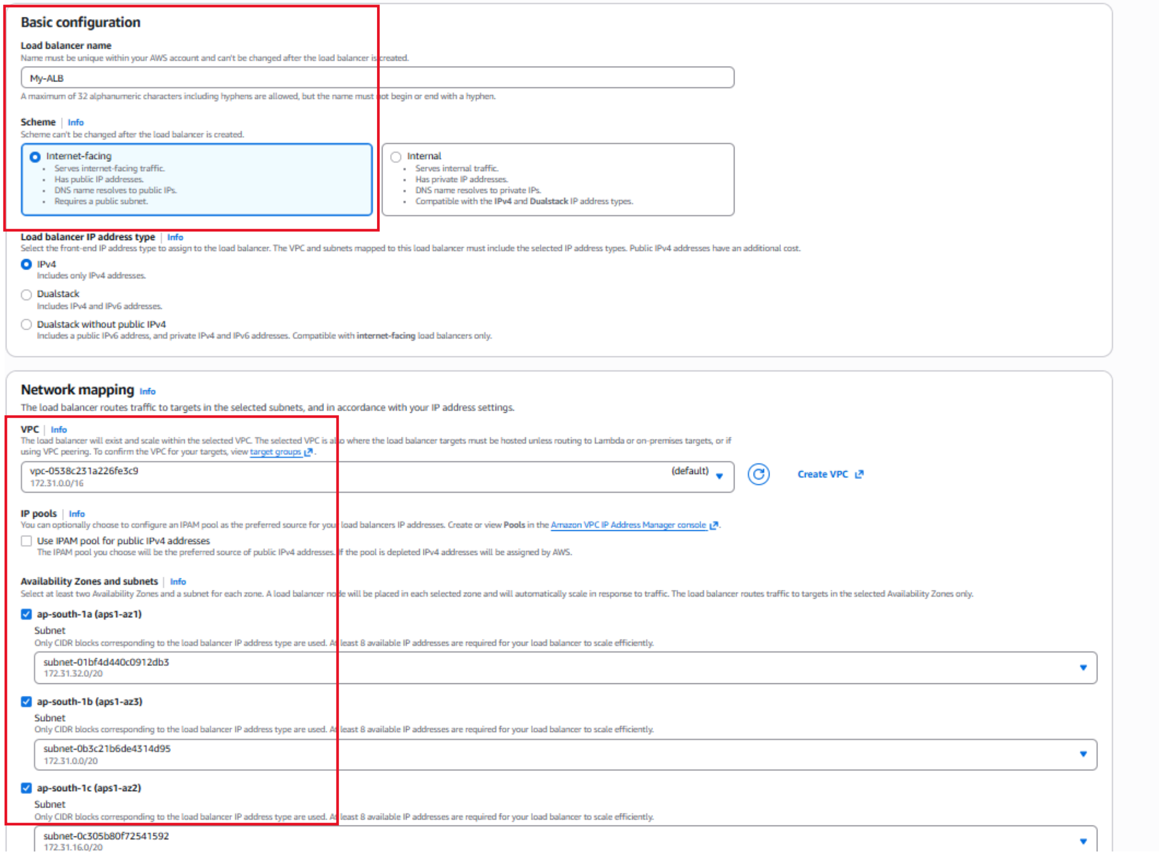
Scheme: **internet-facing**

Network mapping,

Choose: **default VPC**

Availability zone: Choose all the three

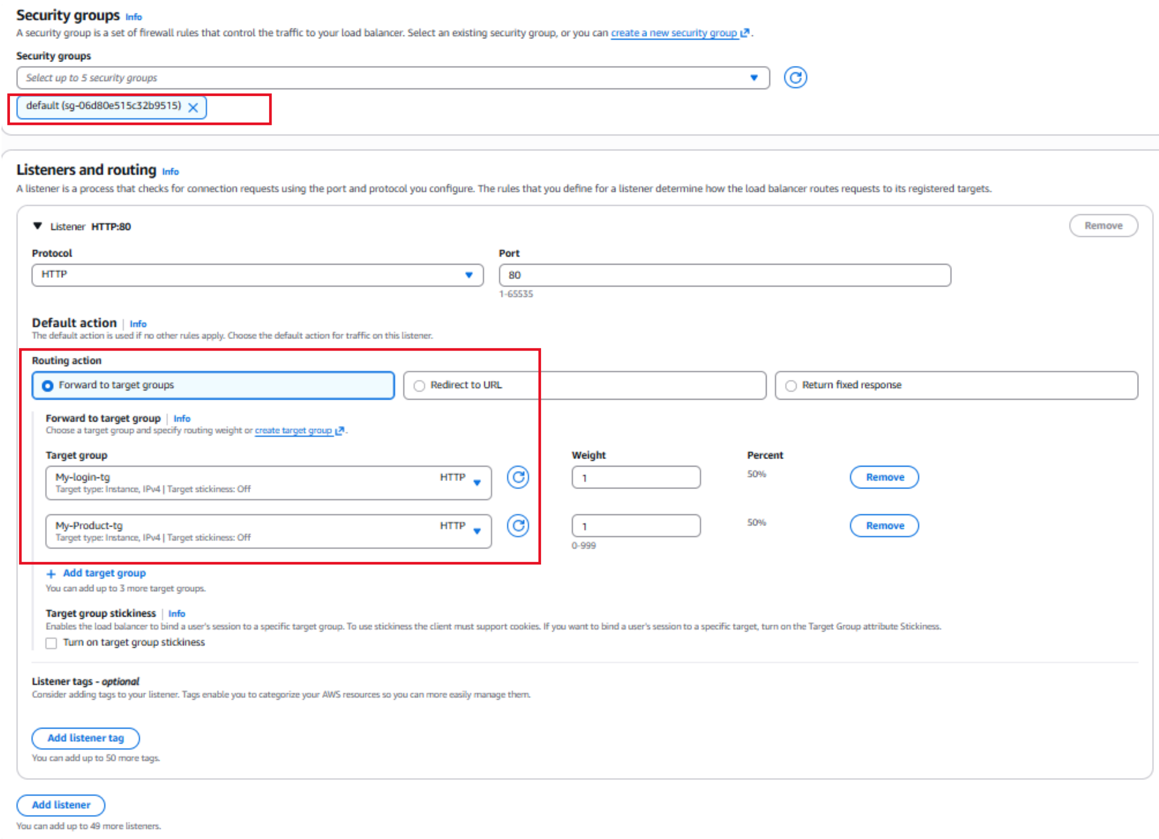
(the correct zone of the EC2 is enough, for more availability we are choosing all the three.)



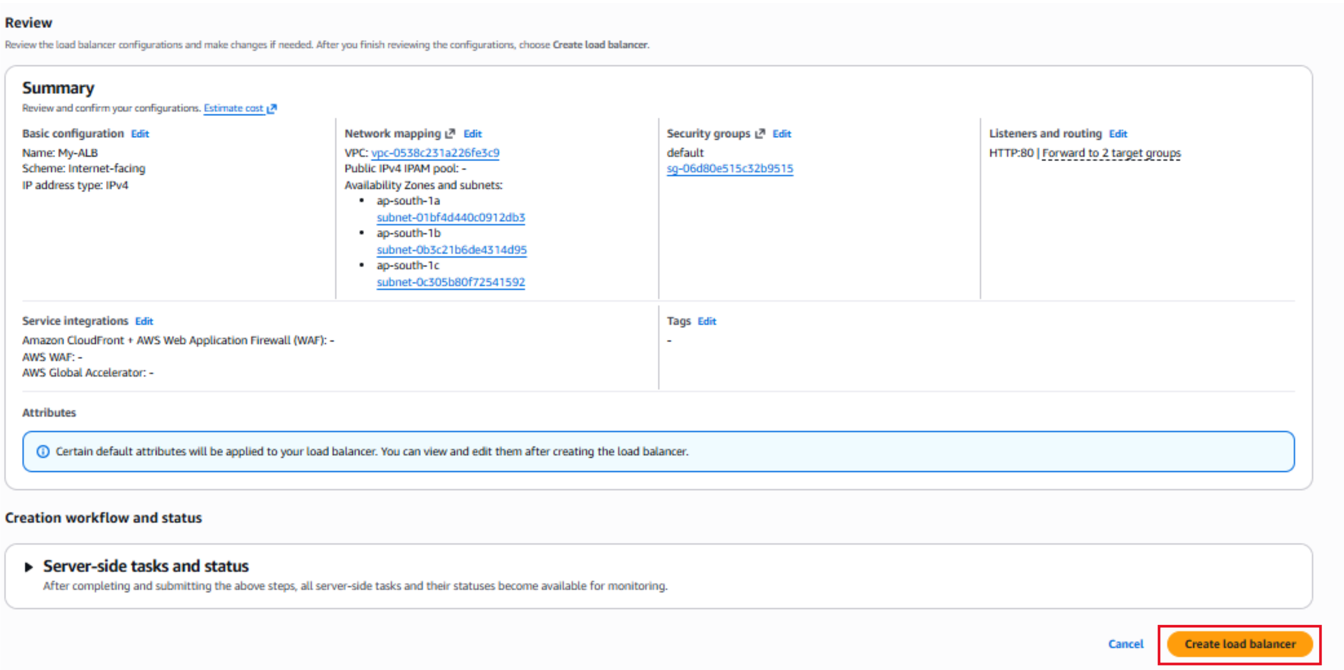

Security group: **default security group**

Default action: **Forward to target groups**

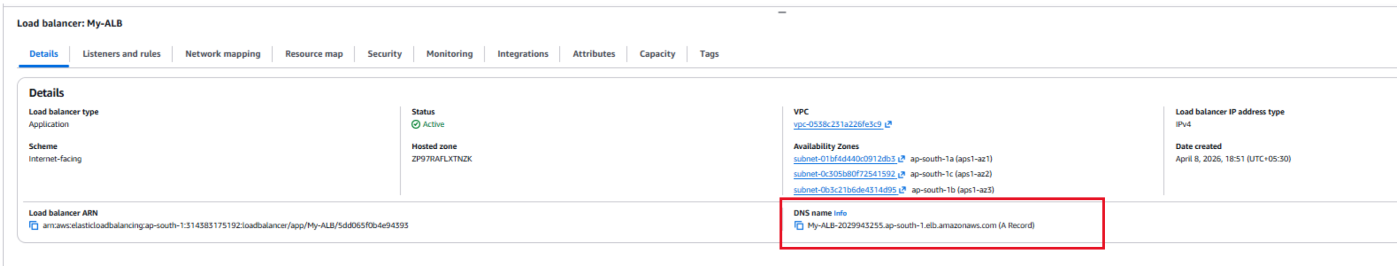
Target group: **My-login-tg → Add → My-Product-tg**



Review the configurations →Create Load Balancer.




Once created → Copy the **DNS of the Load balancer.**




Paste the **DNS** in a new Private Browser, we can see the output.